

web application basics

http

Hypertext Transfer Protocol

http messages

HTTP Messages - packets of data exchanged between client and web server

- **HTTP Requests** - sent by user, trigger actions on web app
- **HTTP Responses** - sent by server in response to the user's request

HTTP Request

```
POST /login HTTP/1.1
Host: tryhackme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 43

username=aleksandra&password=securepassword
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 56
Date: Wed, 29 Aug 2024 12:00:00 GMT

{
  "message": "Login successful!",
  "status": "success"
}
```

HTTP Request

```
POST /login HTTP/1.1
Host: tryhackme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 43

username=aleksandra&password=securepassword
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 56
Date: Wed, 29 Aug 2024 12:00:00 GMT

{
  "message": "Login successful!",
  "status": "success"
}
```

Start Line - introduction (Request or Response)

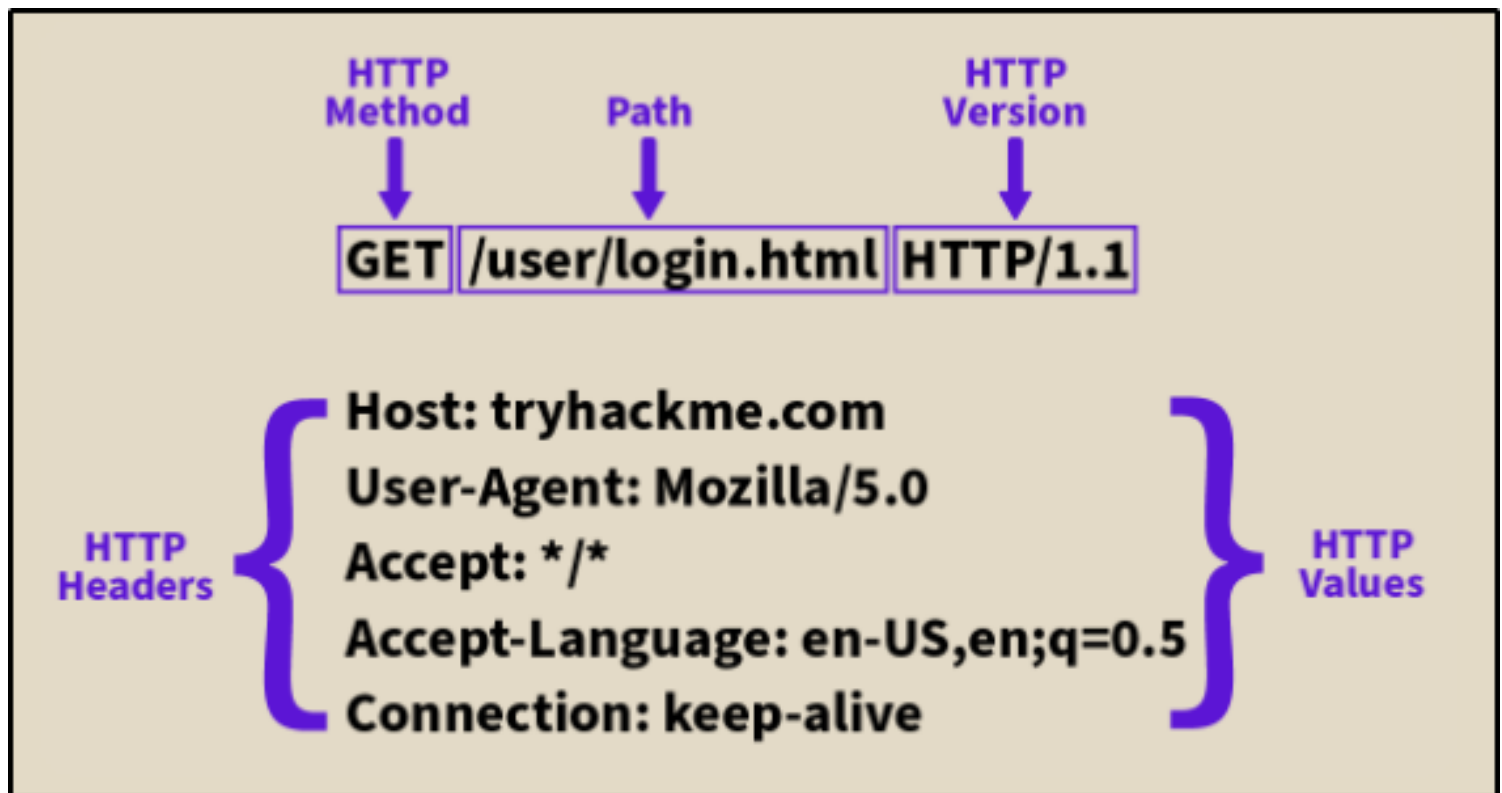
Headers - made up of key-value pairs that provide extra info abt the message (security, content type, etc.)

Empty Line - divider that separates header from the body

Body - where actual data is stored, might include data the user wants to send to the server, it's also where the server puts the content that the user requested (webpage or API data)

http requests

HTTP Request - what a user sends to a web server to interact with a web app and make smth happen



Request Line / Start Line - HTTP method, URL path, HTTP version

Example: [METHOD] / [path] HTTP/[version]

GET /user/login.html HTTP/1.1

• **HTTP Methods** - tells the server what action the user wants to perform on the resource

- **GET** - fetch data without making any changes
- **POST** - sends data to the server (to create or update something)
- **PUT** - replaces or updates something on the server
- **DELETE** - removes something from the server
- PATCH - updates part of a resource
- HEAD - works like GET but only retrieves headers
- OPTIONS - tells you what methods are available for a specific resource
- TRACE - similar to OPTIONS, often for debugging
- CONNECT - to create a secure connection like for HTTPS

• **URL Path** - tells the server where to find the resource the user is asking for

Example: **https://tryhackme.com/api/users/123** (path **/api/users/123** identifies a specific user)

• **HTTP Version** - protocol version

- HTTP/0.9 - first version, only GET reqs
- HTTP/1.0 - added headers and diff types of content, improves caching
- HTTP/1.1 - persistent connections, chunked transfer encoding, better caching, widely used today
- HTTP/2 - multiplexing, header compression, prioritisation for faster perf
- HTTP/3 - built on HTTP/2, uses QUIC protocol for quicker and more secure connections

Many systems still use HTTP/1.1 bc it's well supported and works with most existing setups.

Request Headers - extra info to be conveyed to the web server abt the request

- **Host** - name of the web server the request is for
Host: tryhackme.com
- **User-Agent** - info abt the web browser the request came from
User-Agent: Mozilla/5.0
- **Referer** - URL from which the request came from
Referer: https://www.google.com/
- **Cookie** - where the info the web server previously asked the browser is held
Cookie: user_type=student; room=introtowebapplication; room_status=in_progress
- **Content-Type** - what type or format of data is in the request
Content-Type: application/json

◇

Request Body - in POST and PUT requests, data is located inside here after it is sent to the web server as a request. The format of the data are commonly **URL Encoded**, **Form Data**, **JSON**, or **XML**. • **URL Encoded (application/x-www-form-urlencoded)**

- data is structured in pairs of key and value where **key=value** Multiple pairs are separated by an (&) symbol like **key1=value1&key2=value2**. Special characters are percent-encoded. ◇

```
POST /profile HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

name=Aleksandra&age=27&country=US
```

◇ • **Form Data (multipart/form-data)** - used to send binary data, when uploading files or images to a web server. It allows multiple data blocks to be sent where each block is separated by a **boundary string**. The **boundary string** is the defined header of the request itself.

◇

```
POST /upload HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="username"

aleksandra
----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="profile_pic"; filename="aleksandra.jpg"
Content-Type: image/jpeg

[Binary Data Here representing the image]
----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

- **JSON (application/json)** - data can be sent using the JSON (JavaScript Object Notation) structure. data is formatted in pairs of **name : value**. Multiple pairs are separated by commas, all contained within curly braces { }

```
POST /api/user HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 62

{
  "name": "Aleksandra",
  "age": 27,
  "country": "US"
}
```

- **XML (application/xml)** - data is structured inside labels called **tags** < >, which have an opening and closing. These labels can be nested within each other.

```
POST /api/user HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/xml
Content-Length: 124
```

```
<user>
  <name>Aleksandra</name>
  <age>27</age>
  <country>US</country>
</user>
```

http responses

HTTP Response - when you interact with a web app, server sends back an HTTP response to say if the request was successful or not. These include a **status code** and a short explanation called **Reason Phrase** that gives insight into how the server handled the request.



Status Line - first line, gives 3 key pieces of info:

- **HTTP Version** - which version of HTTP is being used
- **Status Code** - 3-digit number showing the outcome of the request
- **Reason Phrase** - short explanation of the status code, human-readable

◇ **Status Code and Reason Phrases**

◇ • **(100-199) Informational Responses** - the server received part of the request and is waiting for the rest, "keep going" signal.

• **(200-299) Successful Responses** - everything worked as expected, server processed the request and sent back requested data.

• **(300-399) Redirection Messages** - the resource you requested has moved to a different location, usually providing a new URL.

◇ • **(400-499) Client Error Responses** - indicate a problem with the request. Maybe URL is wrong, or missing required info like authentication.

• **(500-599) Server Error Responses** - server encountered an error while trying to fulfill the request. Usually server-side issues and not the client's fault.

◇ **Common Status Codes:**

- ◇ • **100 (Continue)** - server got the first part and is ready for the rest
- **200 (OK)** - request was successful and server is sending back the requested resource
- **301 (Moved Permanently)** - requested resource has been permanently moved to a new URL
- **404 (Not Found)** - server couldn't find the resource at the given URL
- **500 (Internal Server Error)** - something went wrong on the server's end and it couldn't process the request

Response Headers - when a web server responds to an HTTP request, it includes HTTP response headers. These are key-value pairs. These headers provide important info abt the response and tell the client how to handle it.

HTTP
Version

Response
Code

HTTP/1.1

200 OK

HTTP
Headers

Content-Type: application/json
Content-Length: 34
Date: Wed, 29 Aug 2024 12:00:00 GMT
{
 "message": "Login successful!",
 "status": "success"
}

HTTP
Values

• **Date** - exact data and time when the response was generated by the server. ◊ ▪ Date: Fri, 23 Aug 2024 10:43:21 GMT

◊ • **Content-Type** - what kind of content the client is getting (HTML, JSON, or smth else). It also includes the character set (like UTF-8) to help the browser display it properly.

◊ ▪ Content-Type: text/html; charset=utf-8

◊ • **Server** - what kind of server software is handling the request, good for debugging, can also reveal server information.

◊ ▪ Server: nginx

▪

◊ **Common Response Headers:**

◊ • • **Set-Cookie** - sends cookies from the server to the client, which client then stores and sends back with future requests. Set cookies with **HttpOnly** flag so they cannot be accessed by JavaScript and **Secure** flag so they are only sent over HTTPS.

▪ - Set-Cookie: sessionId=38af1337es7a8

• • **Cache-Control** - tells the client how long it can cache the response before checking with the server again. **no-cache** prevents sensitive info from being cached if needed.

▪ - Cache-Control: max-age=600

- **Location** - used in redirection (3xx) responses, tells the client where to go next if the resource has moved.
- Location: /index.html
-

Response Body - where the actual data lives (like HTML, JSON, images, etc.) that the server sends back to the client. ◇ ▪

http security headers

HTTP Security Headers - help improve overall security of a web app by providing mitigations against attacks like Cross-Site Scripting (XSS), clickjacking, etc.

- **Content-Security-Policy (CSP)** - additional security layer against XSS. provides a way for admins to say what domains or sources are considered safe.

-
-

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.tryhackme.com; style-src 'self'
```

- ◇
- ◇ • **default-src** - default policy of self, which means only the current website

- **script-src** - policy for where scripts can be loaded from, which is self along with scripts hosted on https://cdn.tryhackme.com

- ◇ ▪ • **style-src** - policy for where style CSS style sheets can be loaded from the current website (self)

- ◇ • **Strict-Transport-Security (HSTS)** - ensures that web browsers will always connect over HTTPS.

- ◇
- ◇

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

◇

- • **max-age** - expiry time in seconds for this setting
- • **includeSubDomains** - optional setting that instructs the browser to also apply this setting to all subdomains
- • **preload** - optional setting that allows website to be included in preload lists.

◇ • • **X-Content-Type-Options** - instruct browsers not to guess the MIME type of a resource but only use the Content-Type header.

◇

X-Content-Type-Options: nosniff

◇ • • **nosniff** - instructs the browser not to sniff or guess the MIME type.

◇ • • **Referrer-Policy** - controls the amount of info sent to the destination web server when a user is redirected from the source web server, such as when they click a hyperlink. The header is available to allow a web admin to control what information is shared.

◇

- **Referrer-Policy: no-referrer**
- **Referrer-Policy: same-origin**
- **Referrer-Policy: strict-origin**
- **Referrer-Policy: strict-origin-when-cross-origin**

• • **no-referrer** - disables any info being sent abt the referrer

• • **same-origin** - will only send referrer info when the destination is part of the same origin. (when you want referrer info passed when hyperlinks are in same website but not outside to external websites)

• • **strict-origin** - only sends the referrer as the origin when the protocol stays the same. a referrer is sent when an HTTPS connection goes to another HTTPS connection.

• • **strict-origin-when-cross-origin** - similar to strict-origin except for same-origin requests, where it sends the full URL path in the origin header.

◇

◇

overview

Front End

- **HTML (Hypertext Markup Language)** - foundation of web apps (instructs what to display and how to display it)

<pre><!DOCTYPE html> <html> <body> <h1>My First Heading</h1> <p>My first paragraph.</p> </body> </html></pre>	My First Heading My first paragraph.
--	--

- **CSS (Cascading Style Sheets)** - design/style or appearance of a web app

<pre><!DOCTYPE html> <html> <head> <style> body { background-color: lightblue; } h1 { color: white; text-align: center; } p { font-family: verdana; font-size: 20px; } </style> </head> <body> <h1>My First CSS Example</h1> <p>This is a paragraph.</p> </body> </html></pre>	My First CSS Example This is a paragraph.
--	---

- **JS (JavaScript)** - more complex, allows choices and decisions to be made on what to display

<pre><!DOCTYPE html> <html> <body> <h1>My First JavaScript</h1> <button type="button" onclick="document.getElementById('demo').innerHTML = Date()"> Click me to display Date and Time.</button> <p id="demo"></p> </body> </html></pre>	My First JavaScript Click me to display Date and Time. Wed Jun 10 2026 23:45:17 GMT+0800 (Philippine Standard Time)
--	--

Back End

- **Database** - where info can be stored
- **Infrastructure** - web servers, application servers, storage, networking devices, etc.
- **Web Application Firewall** - optional, filters out dangerous requests

practical task

GET request to /api/users. What is the flag?•

GET

https://tryhackme.com/api/users

Go

GET api/users HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 9

country=US

Response

HTTP/1.1 200 Ok
Server: nginx/1.15.8
Thu, 11 Jun 2026 2 59 21 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 633
Last-Modified: Thu, 11 Jun 2026 2 59 21 GMT

<html>
<head>
 <title>TryHackMe</title>
</head>
<body>
 <table class="table-auto"><thead><tr class="bg-gray text-white"><th class="w-20">Name</th><th class="w-20">Age</th><th class="w-20">Country</th><th>Flag</th></tr></thead><tbody><tr><td class="text-center">Alice</td><td class="text-center">28</td><td class="text-center">US</td><td class="text-center"></td></tr><tr><td class="text-center">Bob</td><td class="text-center">34</td><td class="text-center">US</td><td class="text-center"></td></tr><tr><td class="text-center">Charlie</td><td class="text-center">25</td><td class="text-center">CA</td><td class="text-center">THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}</td></tr></tbody></table>
</body>
</html>

Flag: **THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}**◊

2. Make a **POST** request to `/api/user/2` and update the **country** of Bob from **UK** to **US**. What is the flag?

POST

https://tryhackme.com/api/user/2

Go

POST api/user/2 HTTP/1.1

Host: tryhackme.com

User-Agent: Mozilla/5.0 Firefox/87.0

Content-Length: 9

country=US

Response

HTTP/1.1 200 Ok

Server: nginx/1.15

Thu, 11 Jun 2026 3

Content-Type: app

Content-Length: 6

Last-Modified: Th

<html>

<head>

<title>TryHackM

</head>

<body>

<table class="table-auto"><thead><tr class="bg-gray text-white"><th class="w-20">Name</th><th class="w-20">Age</th><th class="w-20">Country</th><th>Flag</th></tr></thead><tbody><tr><td class="text-center">Alice</td><td class="text-center">28</td><td class="text-center">US</td><td class="text-center"></td></tr><tr><td class="text-center">Bob</td><td class="text-center">34</td><td class="text-center">US</td><td class="text-center"></td></tr><tr><td class="text-center">Charlie</td><td class="text-center">25</td><td class="text-center">CA</td><td class="text-center">THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}</td></tr></tbody></table>

</body>

</html>

Parameters

Please add key-value pairs to customize your requests.

Enter key

: Enter value

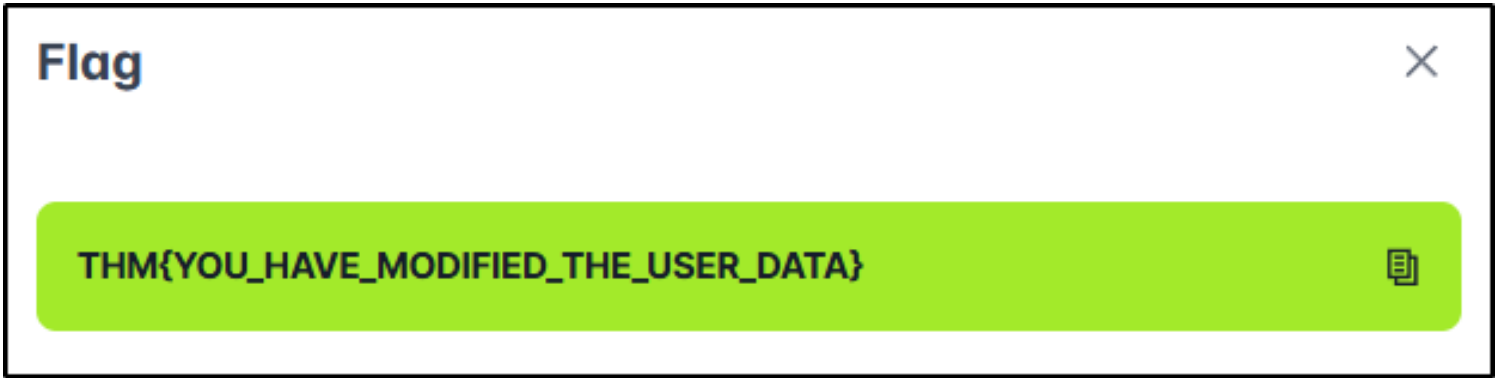


country

: US

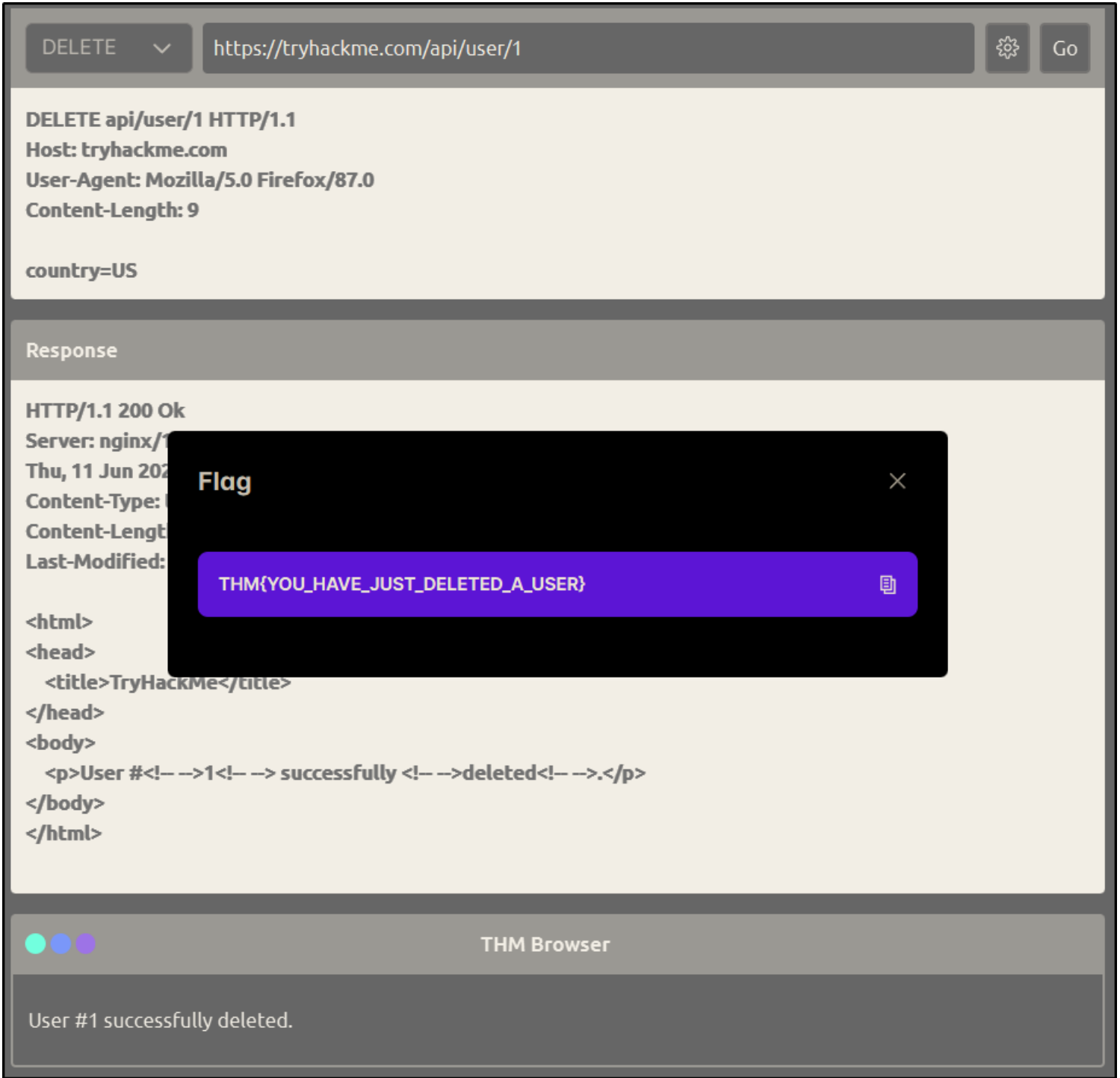


Proceed



Flag: **THM{YOU_HAVE_MODIFIED_THE_USER_DATA}**

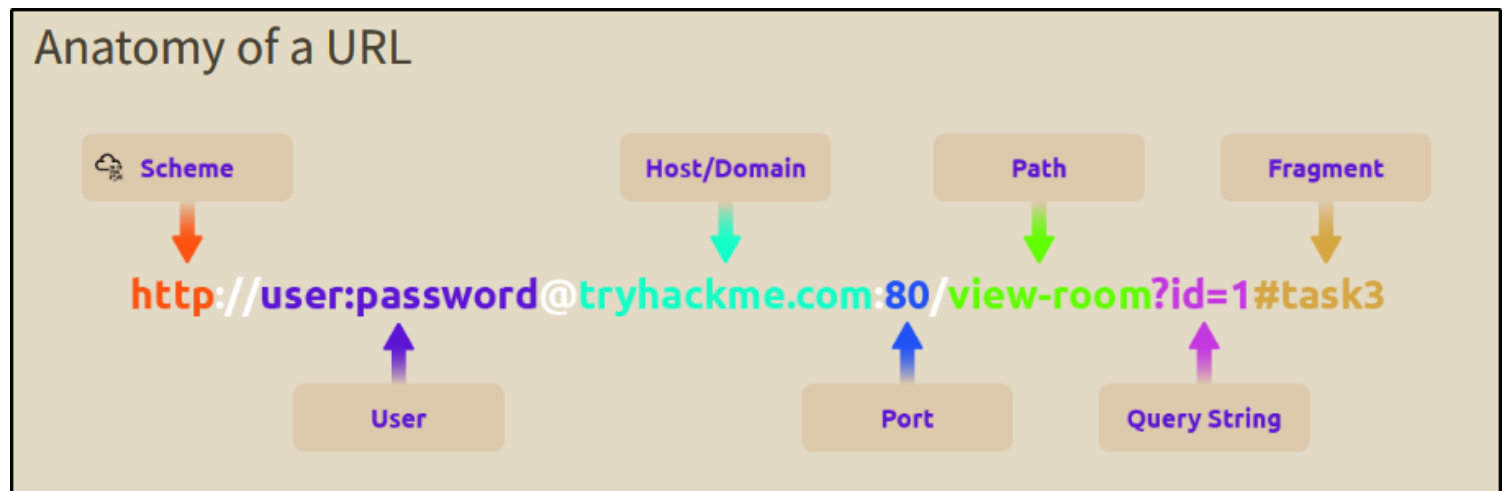
3. Make a **DELETE** request to `/api/user/1` to delete the user. What is the flag?



Flag: **THM{YOU_HAVE_JUST_DELETED_A_USER}**

url

URL (Uniform Resource Locator) - web address



Scheme - protocol used, commonly HTTP or HTTPS (secure)

User - some urls include user's login details

Host/Domain - most important, tells you which website you're accessing • **typosquatting** - fake domains, almost look like the real ones but has small differences

Port - directs your browser to the right service

Path - points to a file or page

Query String - starts with a question mark (?), often for search terms or form inputs ◊ **injections** - malicious code is added

Fragment - starts with hash (#), point to a specific section of a webpage